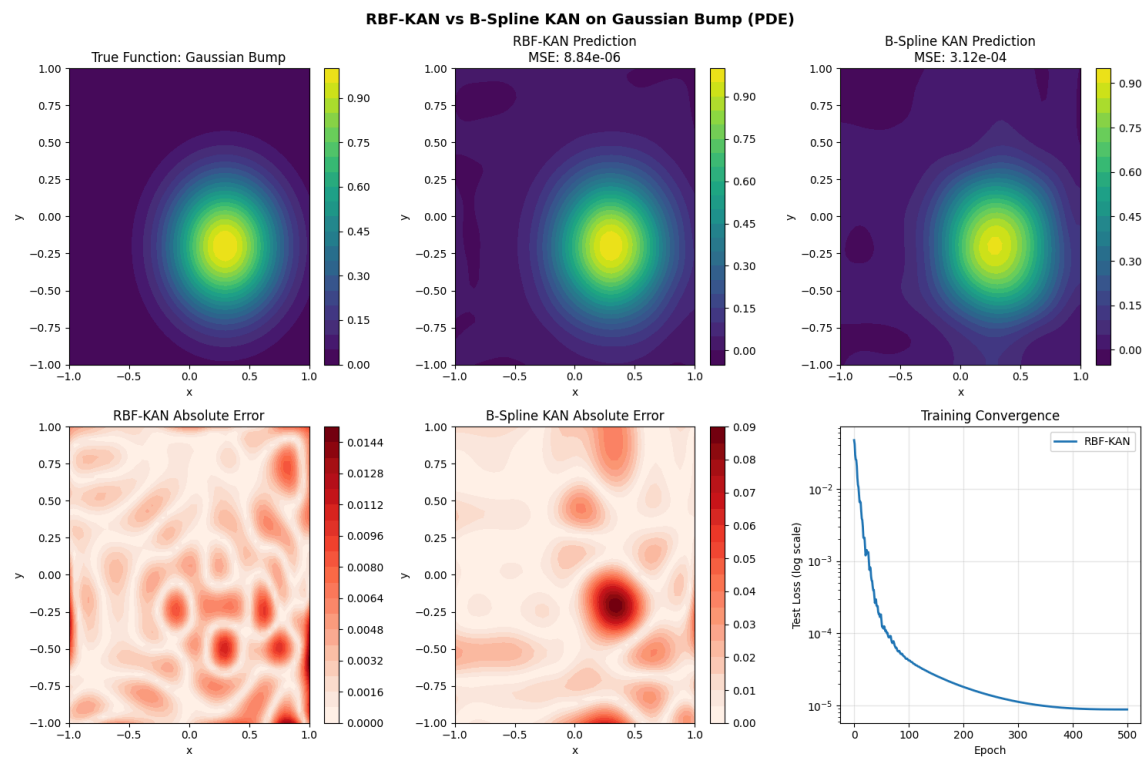
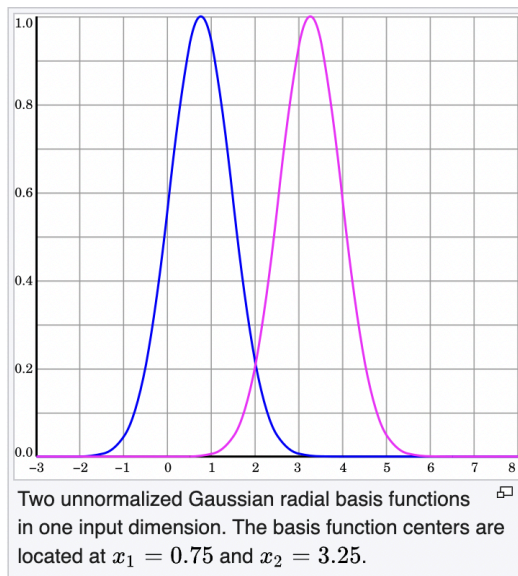
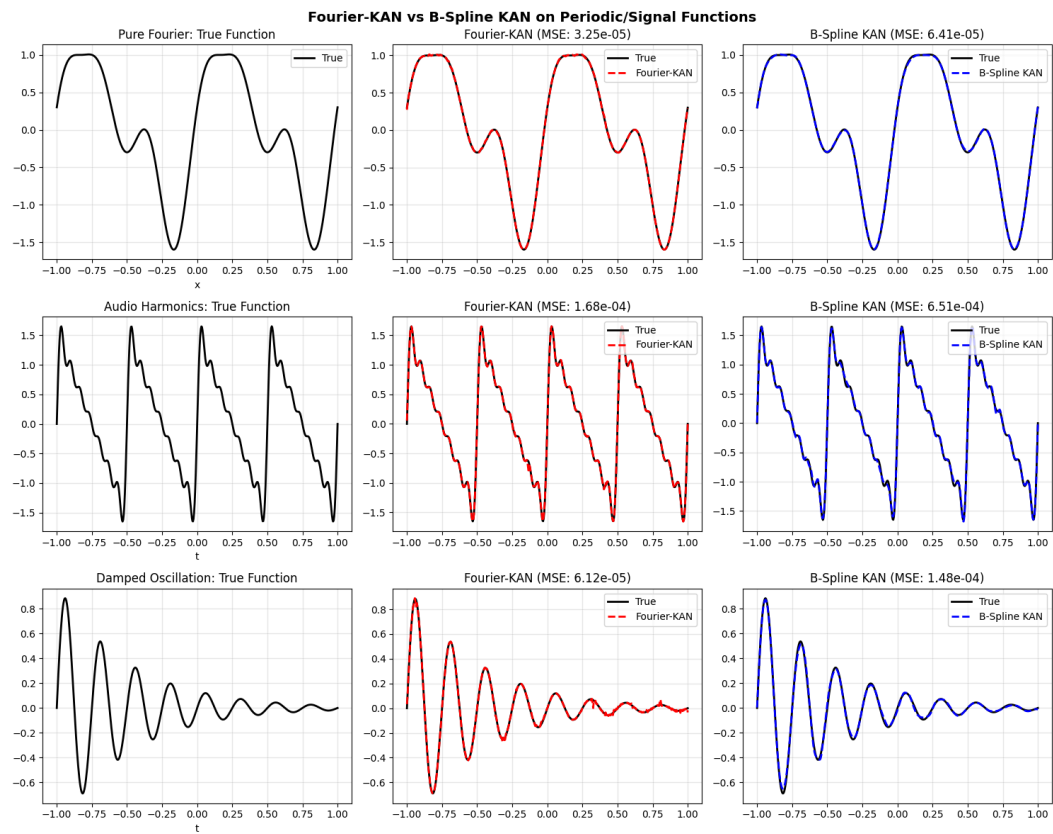


# KAN performance based on different activation functions vs B-Spline basis functions

## 1. Radial basis function (RBF)

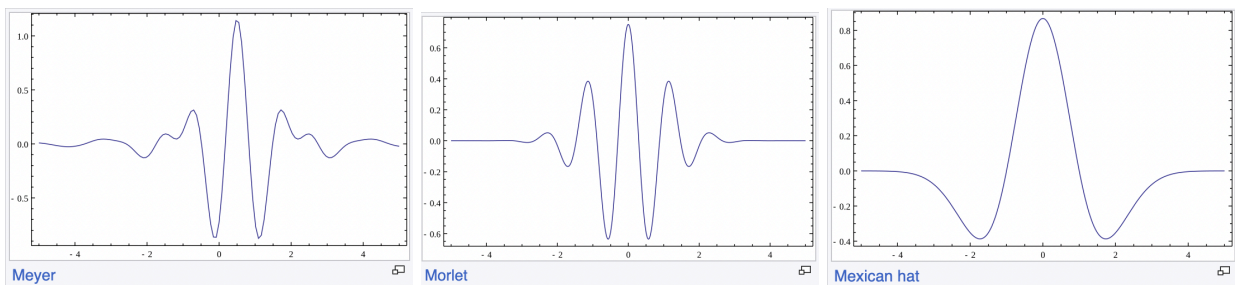


## 2. Fourier transform(frequency based data)

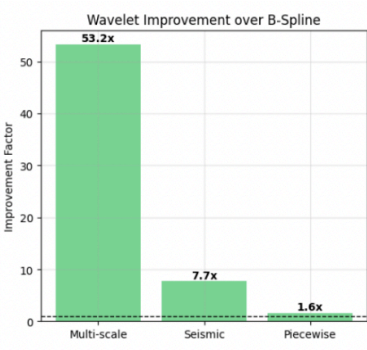
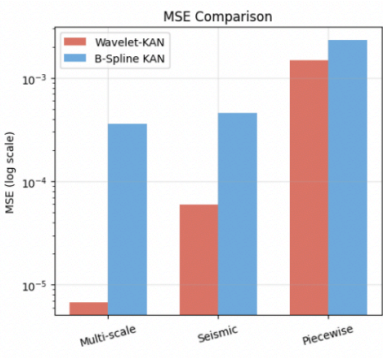
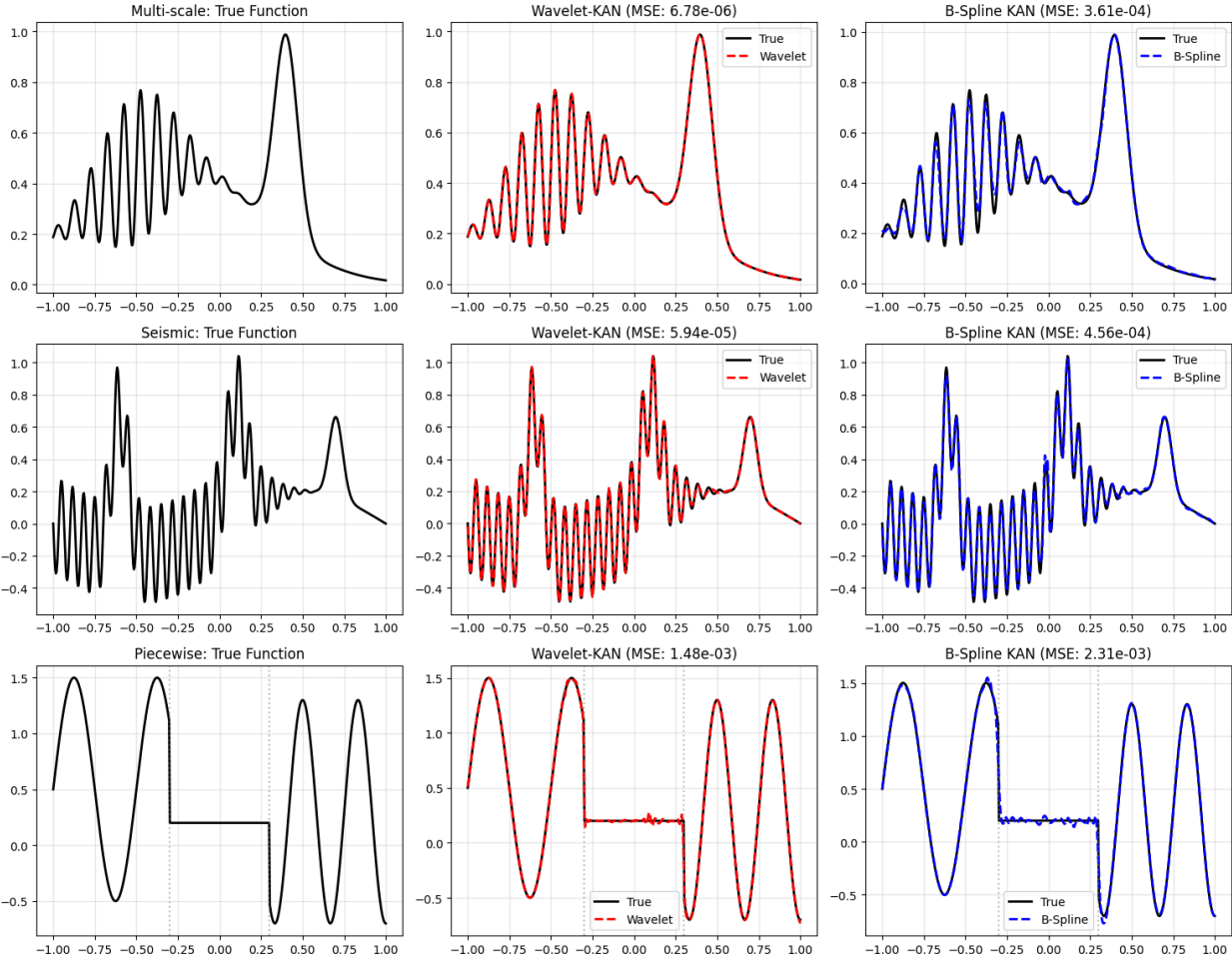


## 3. Wavelet

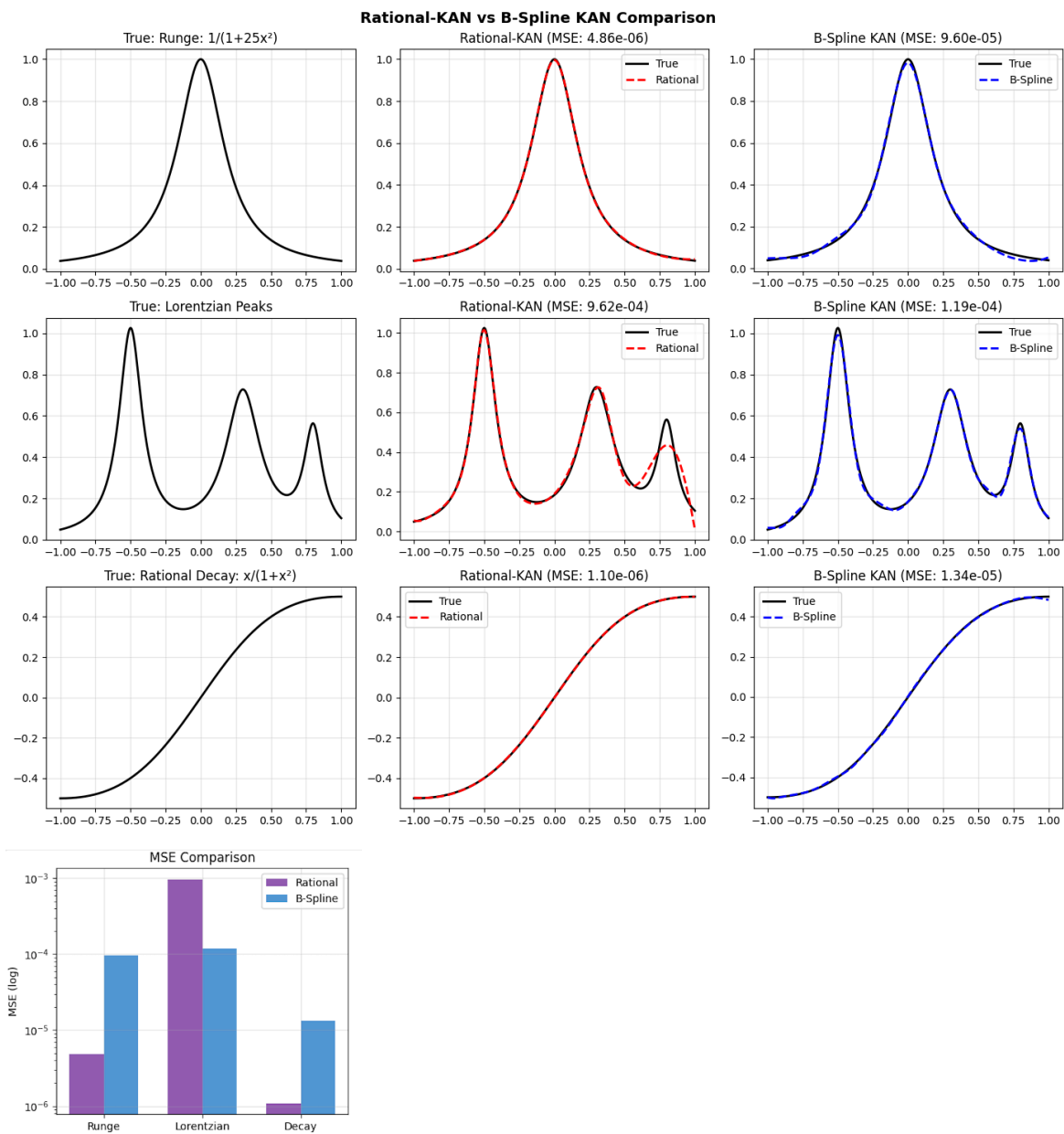
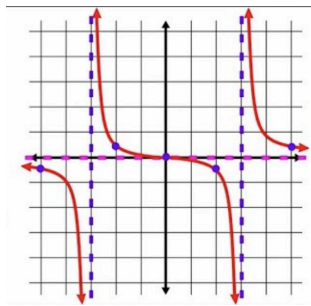
### 3 important Wavelets



Wavelet-KAN vs B-Spline KAN Comparison



4. Rational Function

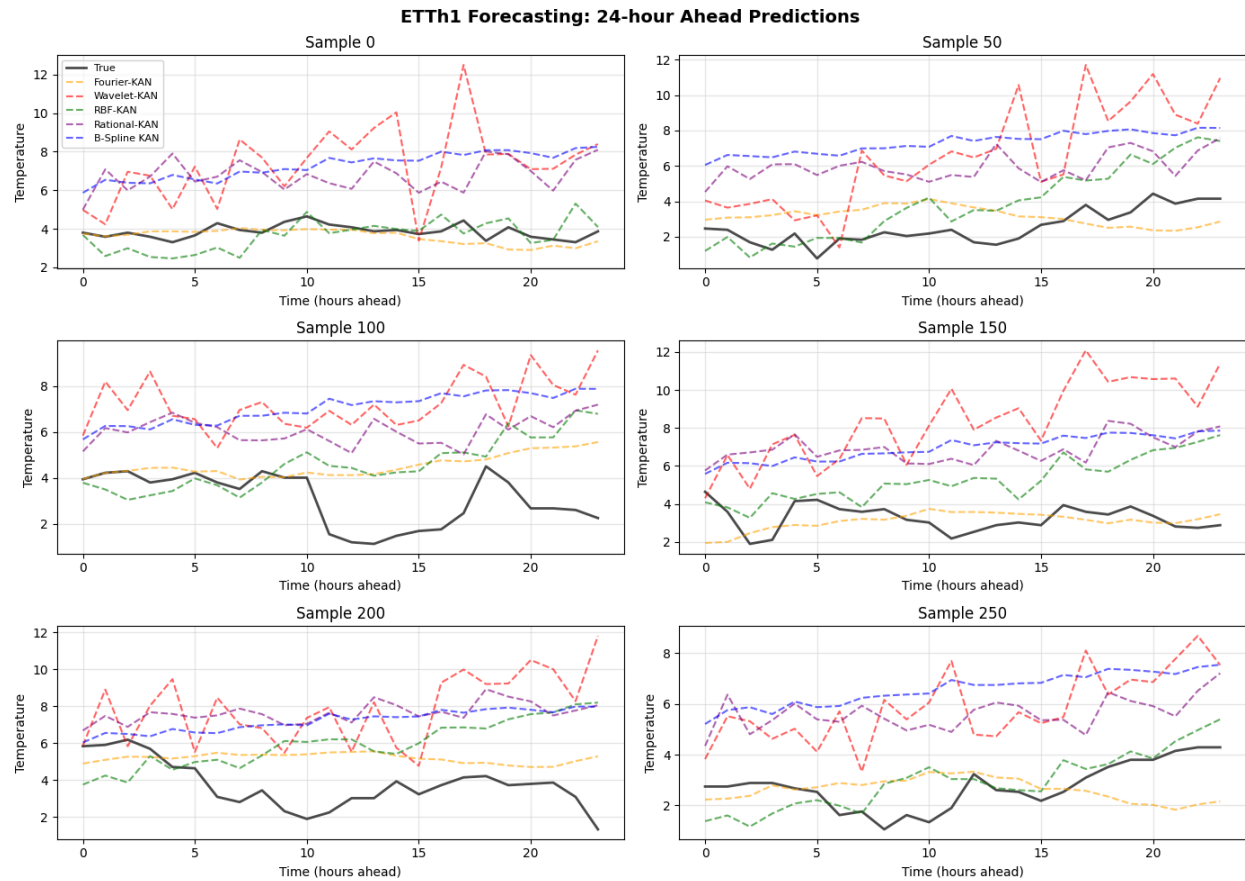


KAN performance with different activation functions on Real world dataset example:  
This task on Time Series Dataset:

**1. ETT = Electricity Transformer Temperature**  
**h1 = Hourly data, Transformer 1**

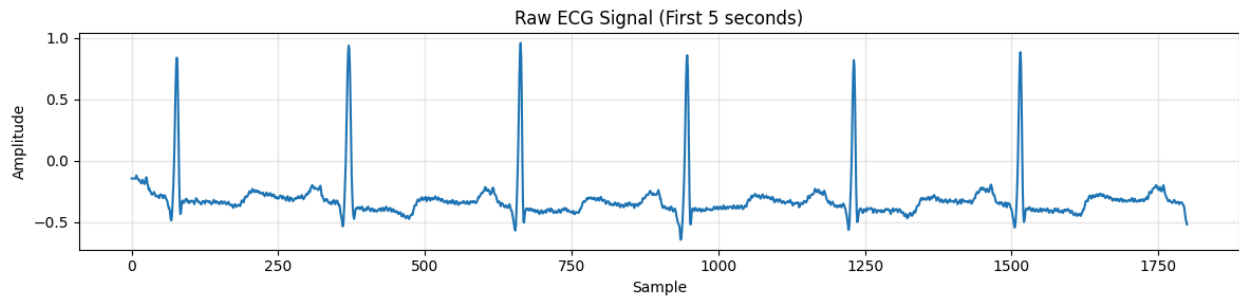
ETTh1 is a real-world time series dataset from a power plant in China that records:

- Electricity load (power consumption)
- Transformer oil temperature (OT)
- Other transformer parameters

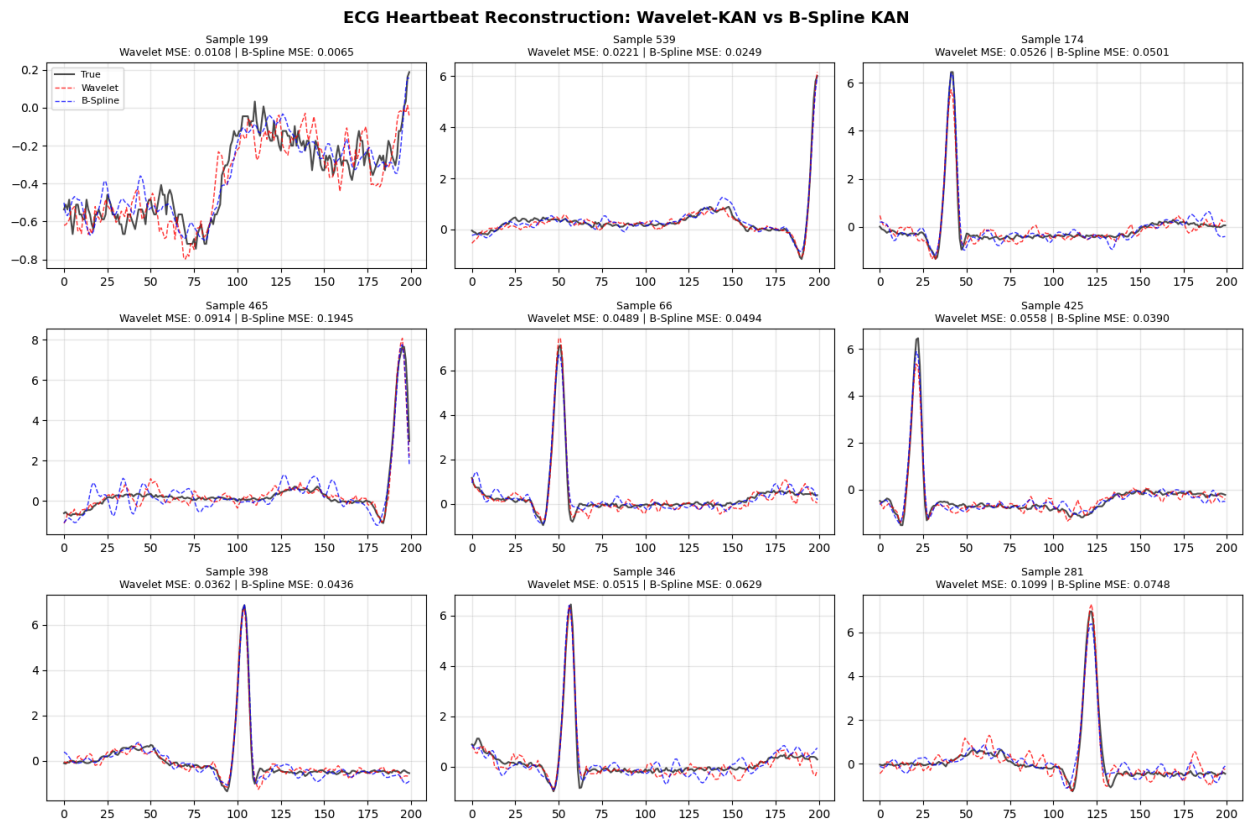


## 2. ECG (Electrocardiogram) = Electrical recording of heart activity

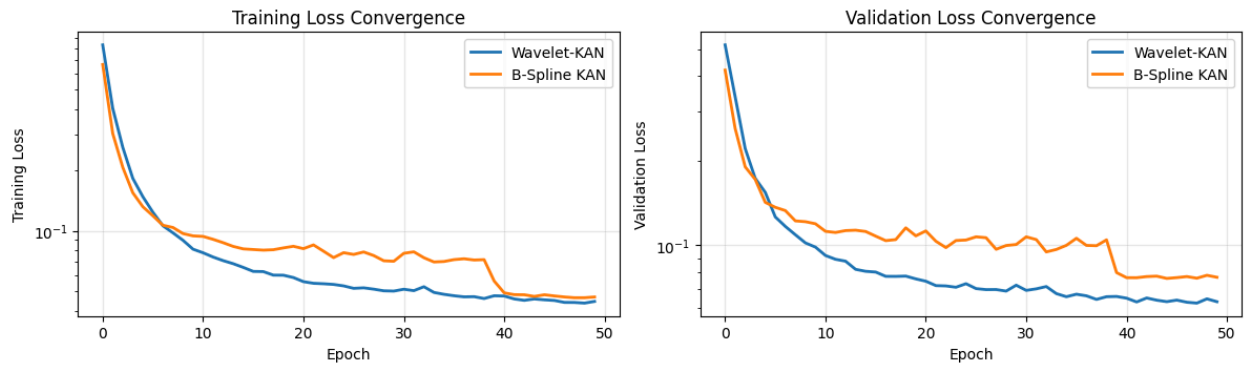
Source: The MIT-BIH (Massachusetts Institute of Technology - Beth Israel Hospital) database



KAN performance:



### Training Dynamics: Wavelet-KAN vs B-Spline KAN



Improvement of network:

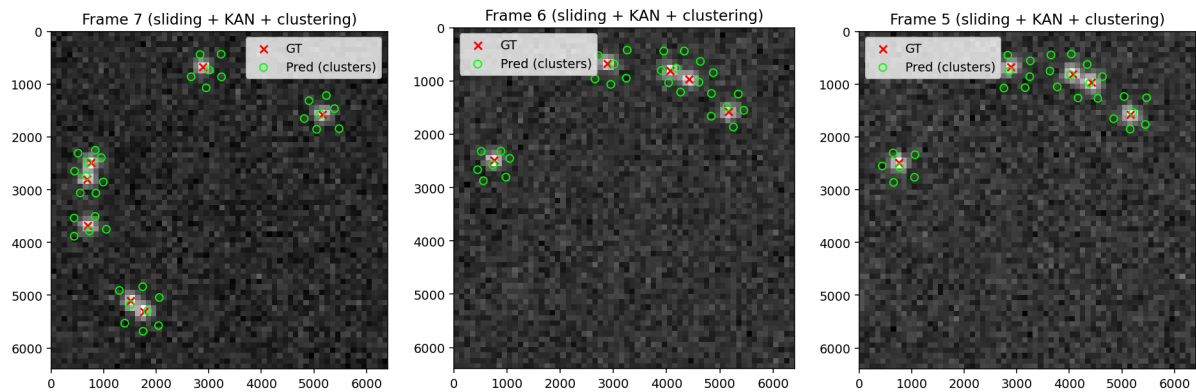
Cropping and stride:

7\*7 - stride 2

Test on 2D\_L\_Guass\_dataset

The network is the same .

Why cluster: Because many overlapping sliding windows see the same emitter, and noise can also affect patches, group nearby candidates into clusters.



My suggestion:

## Two-stage pipeline

Use KAN only for **refinement**, not for detection:

### 1. Classical detection first

- On each 64×64 frame:
  - Smooth with a small Gaussian / LoG.
  - Find **local maxima** in intensity.
- This gives a list of **candidate pixels**, close to #emitters per frame.

### 2. KAN refinement

- For each candidate pixel:
  - Extract one 7×7 patch, normalize.
  - Feed to KAN, get  $(dx, dy)$ .
  - Convert to final  $(x_{nm}, y_{nm})$ .